Project 1: MAGIC TELESCOPE (Supervised Classification Model)

Main Point: Using features given from the telescope (10 features) determine if it is a hardon or a gamma particle

Data Source: https://archive.ics.uci.edu/dataset/159/magic+gamma+telescope

Imblearn. oversampling = in imbalanced data (there is more blue data than red data) we can purposefully undersample (drop some blue data) to make the representation red and blue or equal or we can purposefully oversample the red to make the representation more equal (double/triple count the red data). A popular undersampling technique is TOMEK undersampling (drop majority data points that are very close to minority data points, keep minority data points alive, Pro: does not drop the data to extreme). A popular oversampling technique is SMOTE (finds the nearest neighbor between red data and fills it with more data in between, con: since linear interpolation a lot of linear lines form).

We need this because we have an imbalance in our data (Gamma: 12332, Hadron: 6688)

So should I undersample or oversample?: Comparing between RadomUnder/Oversampler

Supervised vs Unsupervised video: Supervised has training data with labels (the thing I am trying to predict, this is the truth). Supervised ML is trying to mimic an expert to give me an answer, and unsupervised is trying to learn patterns from past data to predict future data.

Discreet labels = classification (1 or 0, Cat or dog Supervised). Clustering (UnSupversied, find new groups)

continuous labels = regression (line of best fit etc … Supervised). Embedding (Unsupervised, data mining)

Train, Validate, Test: 6:2:2 (typically, or other values work). Train model with train data (analyze loss and feedback to the model). Validation data (make sure that the model can handle unseen data, check loss but not feedback into the model, choose the best model w/ lowest loss). Test

data is fed to the best model (final check to see how generalizable the model is, the loss is the final reported performance for this model)

Loss: diff. b/w prediction and true value

Accuracy: How correct it is

Random_State = 42: this allows for the data split to be the same every time it happens. 42 is an easter egg number chosen. To ensure that the same data frame has been split, I created a hash value for the splits and compared them (hash = a unique ID key for an input)

The Following graphs show a probability distribution of each feature between Gamma particles and Hardon particles.

This will be useful for **feature selection** later: if unnecessary features are used in training, the model might train on this data or might make the model run slower. So we remove certain features so that the model 1. Prevents learning from noise 2. Improved accuracy 3. Reduce training time. I used the filter method.

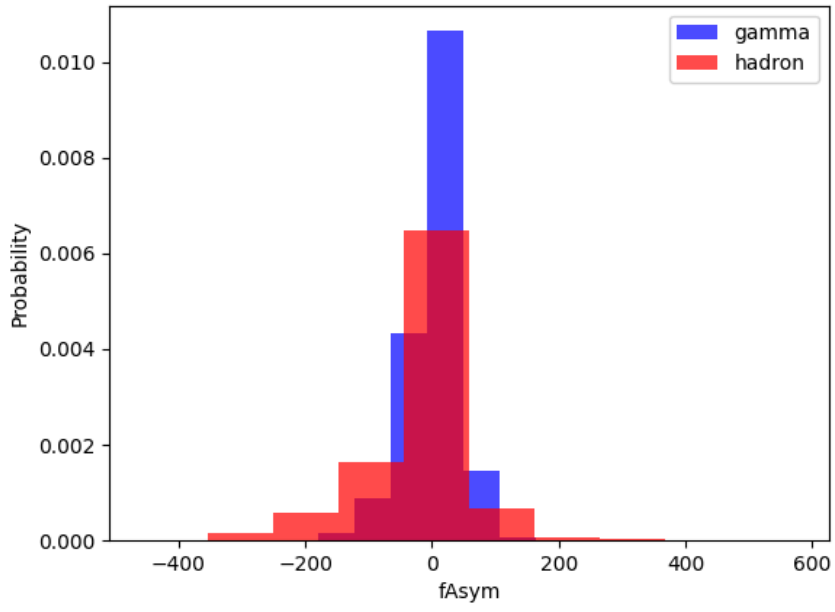


Figure 1.0: Gamma particles have more fAsym feature near or at 0, the standard deviation is much smaller than hadron particles. (gamma std = 39.632629, hadron std = 82.297875)

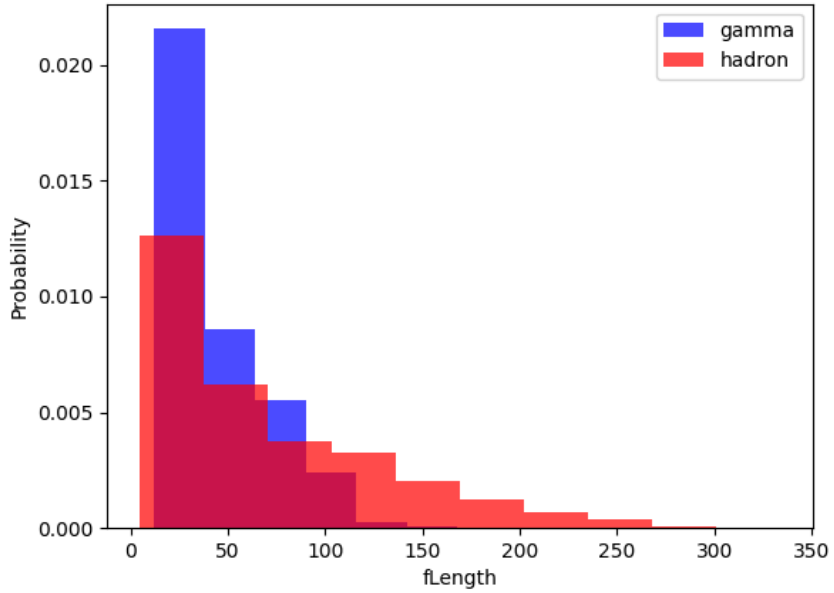


Figure 1.1: Higher probability that gamma particles have lower fLength than hadron particles (gamma std = 26.173434, hadron std = 57.952729)

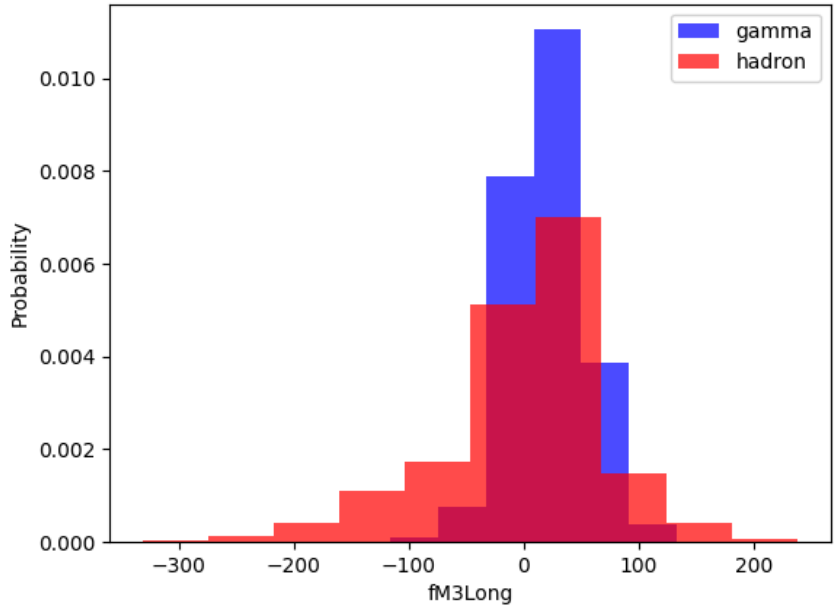


Figure 1.2: Gamma particles have more fM3Long values near or at 0, the standard deviation is much lower than hadron particles (gamma std = 33.942780, hadron std = 70.685797)

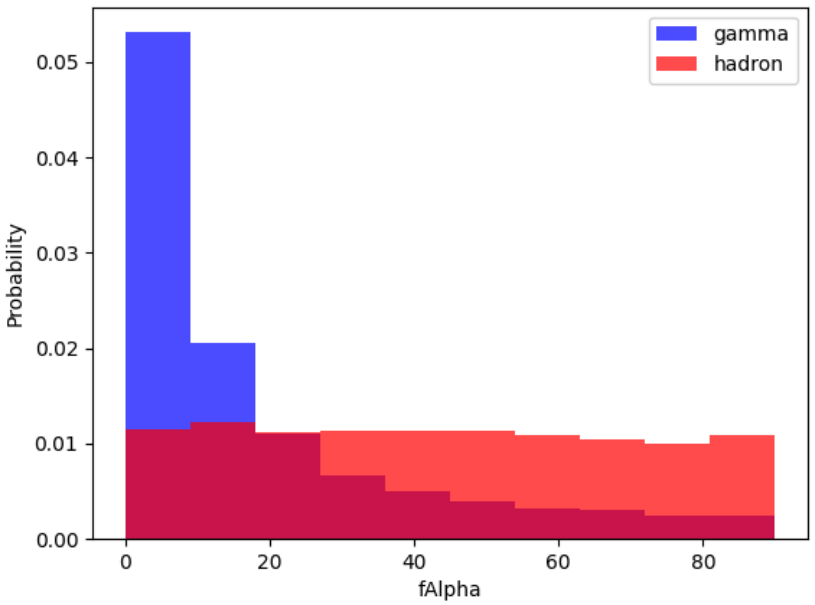


Figure 1.3: Higher probability that gamma particles have lower fAlpha values as to hadron particles having more evenly distributed fAlpha value (gamma std = 21.482585, hadron std = 25.983935)

Scaling a Data Set:

KNN, SVM, and clustering ML models use distance calculations to find the similarity between data points. If a feature has a broad range of values, it might skew the similarity to the larger value one. Also performance-wise, dealing with a bigger number is much more difficult so making them smaller is better (better for convergence).

Standard Scalar: sets mean to 0 and standard deviation to 1.

Classification Report SKlearn

Accuracy: #correct / #total

Precision: #True Positive / #(True Positive + False Positive)

Recall: #True positive / #False negatives

F1-Score: 2 x {(precision x recall)/(precision + recall)}, gives an overall reading for precision & recall. Useful for imbalance classes (than accuracy) because it takes harmonic mean which accounts for the stats of the lower class more.

K-Nearest Neighbour:

Using the distance function (2D: Eculidian), analyze the nearest neighbors of the unknown label to determine what it might be. $d = \sqrt{x^2 + y^2 + z^2 + \ldots}$

Parameters: k (how many neighbors we use to judge the label)

HyperPrameters: whose values that control the learning process

Now we analyze the data (Undersampled vs Oversampled and Feature selection vs no Feature selection)

KNN: Oversampling, No feature selection

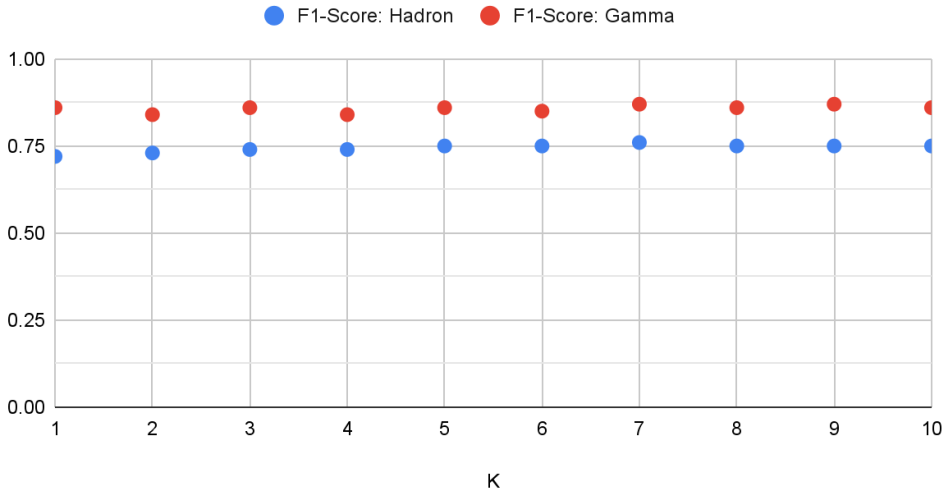


Figure 2.1 Highest F1-Score for both Hadron and Gamma occurred at K=7 (0.76, 0.87 respectively)

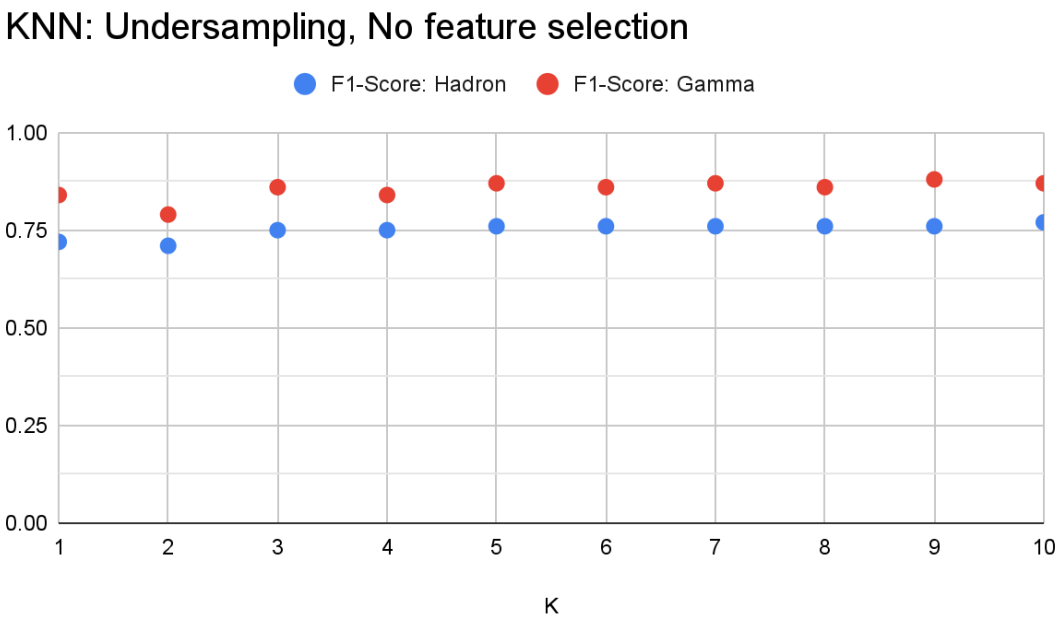


Figure 2.2 Highest F1-Score for Hadron occurred at K=10 (0.77) and for gamma, it occurred at K=9 (0.88)

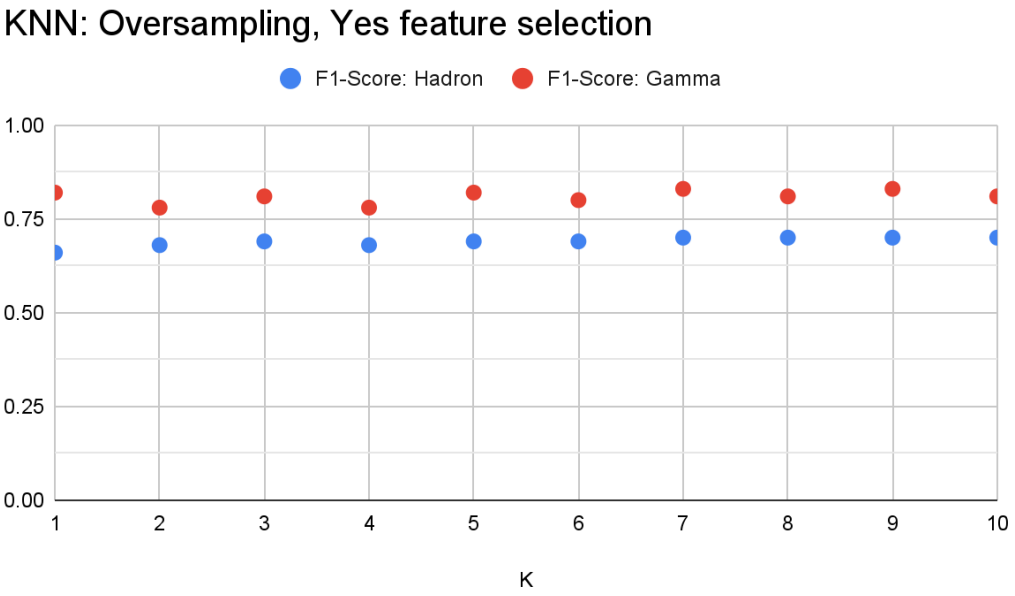


Figure 2.3 Highest F1 score for both Hadron and Gamma particles occur at k=7,9 (0.7 and 0.83 respectively)

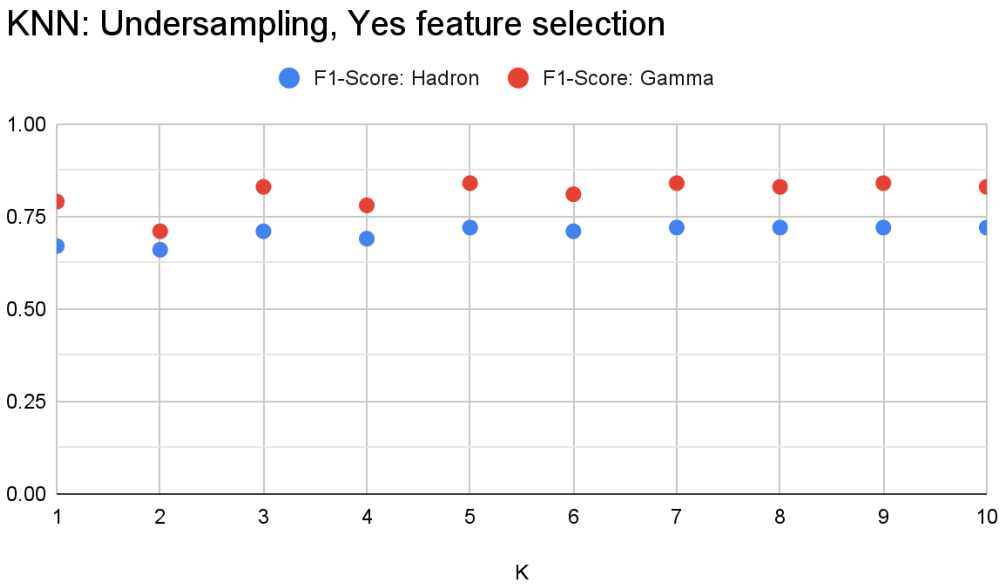


Figure 2.4 Highest F1 score for both Hadron and Gamma particles occur at K=5,7,9 (0.72 and 0.84 respectively)

Analysis

Between oversampled and undersampled F1 scores there wasn't a significant difference. This was probably due to my model using randomOver/Under samplers. The model would probably produce better results with SMOTE or other techniques that combine over/under-sampling.

Between not-feature-selected and feature-selected models, the feature-selected models had a consistently lower F1 score (both over and under-sampled). Based on my research this is because of two main reasons 1. Loss of useful information 2. Overfitting. Loss of useful information can happen when I eliminated what seemed to be features that might have not had a great correlation to gamma or hadron particles, especially with KNN (distance function algorithms) lower dimensionality can harm its distance calculations. Secondly, overfitting may have occurred when the model was trained on data with fewer features. This was probably due to less noise being in the data, noise can make the model more generalized and prevent overfitting.

TEST data results

Not Feature Selected

Classification report for Oversample K=7: F1-score 0.72 0.85 (hadron & gamma)

Classification report for Undersample K=9: F1-score 0.75 0.87 (hadron & gamma)

Classification report for Undersample K=10: F1-score 0.75 0.86 (hadron & gamma)

Feature Selected

Classification report for Oversample K=7: f1-score 0.67 0.82 (hadron & gamma)

Classification report for Oversample K=9: f1-score 0.69 0.83 (hadron & gamma)

Classification report for Undersample K=5: f1-score 0.68 0.83 (hadron & gamma)

Classification report for Undersample K=7: f1-score 0.68 0.83 (hadron & gamma)

Classification report for Undersample K=9: f1-score 0.69 0.84 (hadron & gamma)

In the not feature-selected model undersampling with K=9 had the highest F1-score (0.75 and 0.87) and in feature-selected model undersampling with K=9 had the highest F1-score (0.69 and 0.84). As stated above, the not feature-selected model has a higher overall F1-score. In both cases undersampling with K=9 performed the best.

Naive Bayes:

Recall Bayes Rule (EECS 203): $P(A|B) = \frac{P(B|A)P(A)}{P(B)}$.

We can expand the Bayes Rule to classification and that is the Naive Bayes Method:

$$P(Class_x|Feature\ Vector) = \frac{P(Feature|Class_x)P(Class_x)}{P(Feature)}$$.

The rule for Naive Bayes

$P(C_k|X_1, X_2, \ldots, X_n)\ \alpha\ P(C_k) \prod_{i=1}^{n} P(X_i|C_k)$ ($\prod$ means multiple i from 1 to n)

In English, it means:

the probability that we are in some category (C_k) given

that we have different features (X_n) is proportional to the probability

that of the class in general ($P(C_k)$ times the probability of each of those

features given that we are in this one class that we are testing ($P(X_i|C_k)$

).

So how do we use this?

$y = argmax(P(C_k) \prod_{i=1}^{n} P(X_i|C_k))$ (argmax: will find the value K that will

maximize the expression, MAP: Maximum A Posteriority: pick the K that is most probably to minimize the probability of misclassification)

Assumptions that Naive Bayes uses

Naive Bayes "assumes that **predictors** in a Naïve Bayes model are **conditionally independent**, or **unrelated** to any of the **other features** in the model. It also assumes that **all features contribute equally to the outcome**" (IBM).

Types of Naive Bayes Classifiers

Gaussian Naive Bayes: uses Gaussian distributions (normal distribution) and continuous variables. Fitted by finding the mean and standard deviation

of each class. **Pro:** handles continuous data well, is computationally efficient, and requires less training data. **Con:** Assumes normal distribution (which might not always be true), performs poorly if not true.

Multinomial Naive Bayes: assumes multinomial distributions (useful for discrete data, frequency counts in NLP). **Pro:** simple to implement and interpret, effective for text data and document classification. **Con:** assumes independence of features, not suitable for continuous data.

Bernoulli Naive Bayes: used with Boolean variables (1,0 true/false). **Pro:** works well with binary/boolean data, simple and fast to implement. **Con:** limits applicability to other types of data.

General Pro/Cons: Pro: Naive Bayes is considered a simpler classifier, scales well (efficient), and can handle high dimensional data. **Con:** assumes conditional independence, when the categorical variable does not exist within the training set, the probability of this would be 0 (Zero frequency)

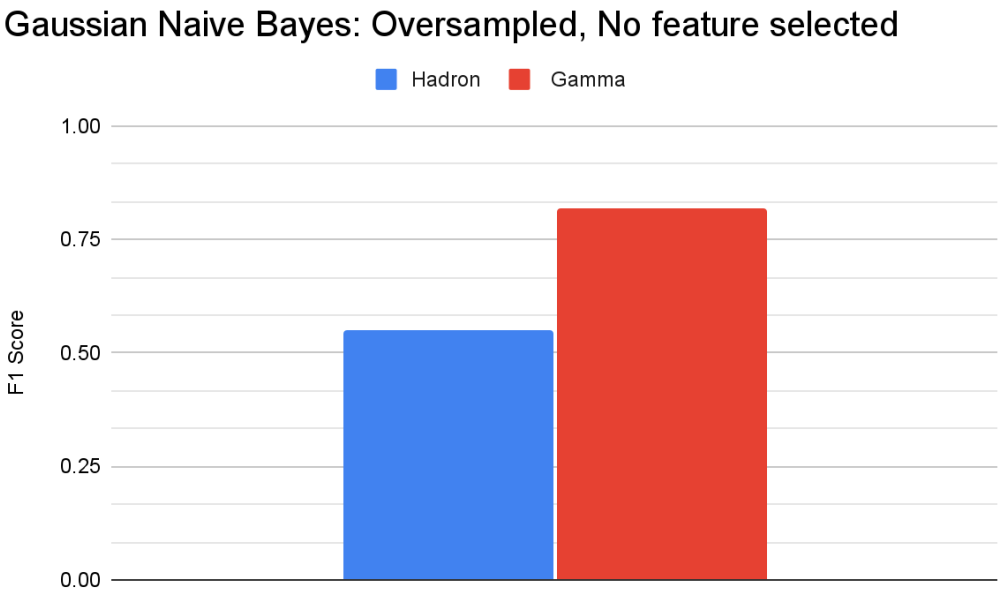


Figure 3.0 F1 score of Hadron is 0.55 and Gamma is 0.82

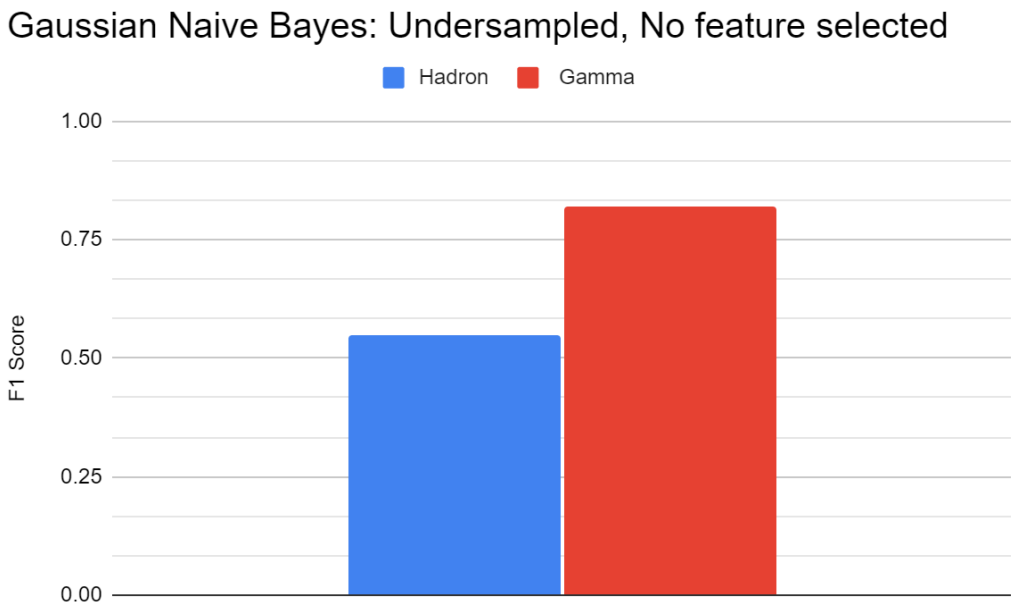


Figure 3.1 F1 score of Hadron is 0.55 and Gamma is 0.82

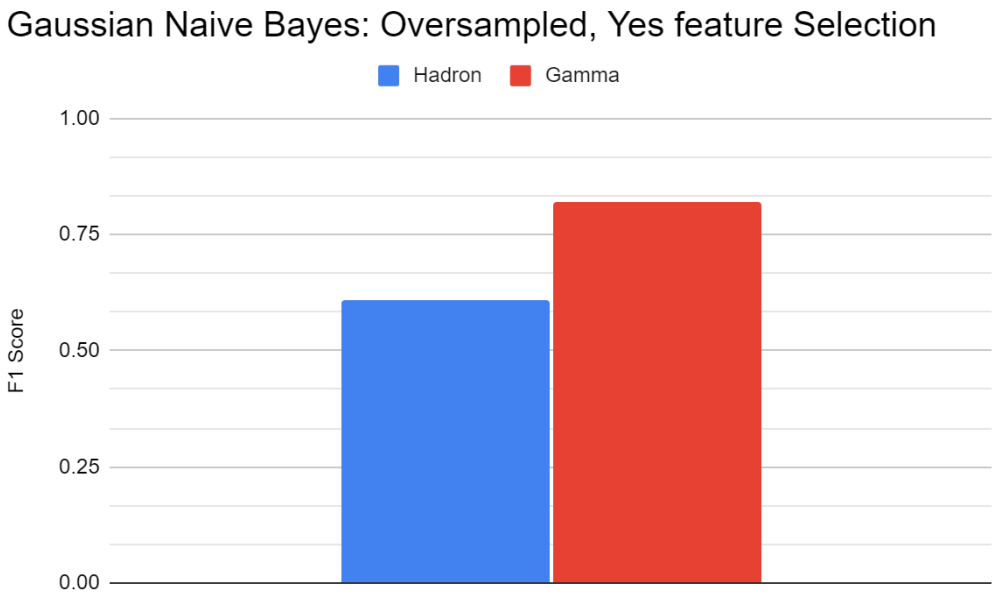


Figure 3.2 F1 score of Hadron is 0.61 and Gamma is 0.82

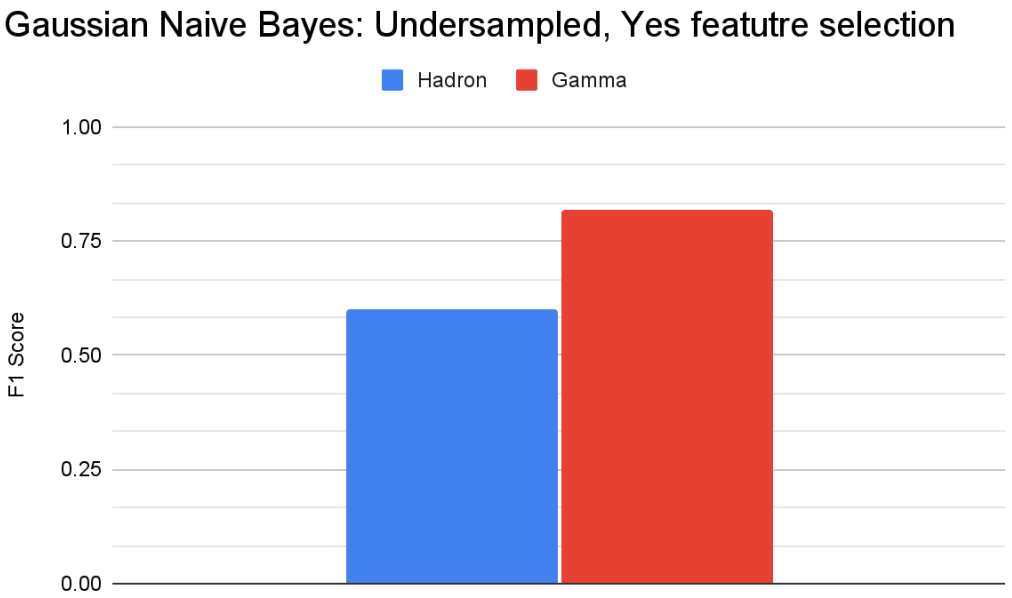


Figure 3.3 F1 score of Hadron is 0.60 and Gamma is 0.82

Analysis

F1 scores of undersampled and oversampled data have not shown any difference.

F1 scores of feature-selected data set performed better for Hadron than not feature-selected data. According to research, this is likely due to the selected features being aligned more with Gaussian distribution. As the Gaussian Naive Bayes model assumes the data to have a normal distribution, the closer the data set is to a normal distribution better the performance of the model would be. As seen above 2 of the selected features showed a close normal distribution. Another potential reason could be the reduction of violations in the assumption of independence. As Naie Bayes assumes each feature is independent to the class, reducing features could have eliminated features that have dependence on each other.

TEST data results

Feature Selected and Oversampled model performance:

Hadron F1 Score: 0.58

Gamma F1 Score: 0.81

Logistic Regression:

How do we model probability (0 or 1)?

From this: $ln(\frac{p}{1-p}) = mx + b$

We get:

$P = \frac{1}{1+e^{-(mx+b)}}$ (Sigmoid Function: $S(x) = \frac{1}{1+e^{-x}}$)

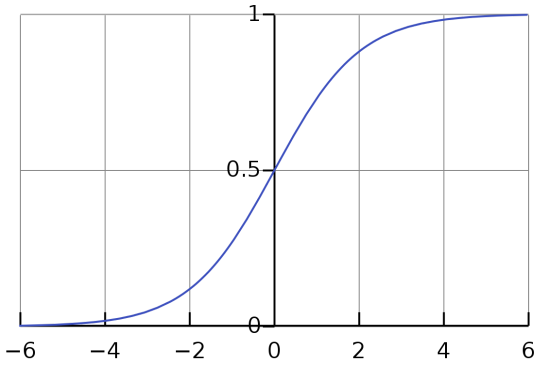


Figure 4.0 Sigmoid Function (max 1, min 0)

What logistic regression does is try to fit our features to the class using the sigmoid function. When we have multiple features it is called a multiple logistic regression.

Penalties (Regularization):

Techniques used to prevent overfitting by adding a constrain or penalty to the model's complexity (make it less complex) and lower the weights (importance) the model has for each feature.

L1 Lasso Regularization:

Adds penalty equal to the absolute value of the magnitude of the coefficients. This encourages sparsity (less features, not all neurons are connected) in the model by driving some of the coefficients to exactly zero, effectively performing feature selection. L1 penalizes (adds abs(coefficient) to the loss function) the model for having higher weights for features in an absolute function

way (linear method), so, in the end, the model tries to decrease the penalty by driving some of the weights to 0.

Use cases: Useful when certain features are important (effective feature selection).

Formula: $Loss + \lambda \sum_i |w_i|$ (abs val. function)

L2 Ridge Regularization:

Adds penalty equal to the square of the magnitude of the coefficients. L2 penalizes larger weights more severely giving each feature different weights (less sparse weights as no feature goes to 0).

Formula: $Loss + \lambda \sum_i w_i^2$ (quadratic function)

Use cases: Useful when there are a lot of correlated features and need to keep them all in the Model but at the same time prevent overfitting (reducing their impact).

Elastic Net Regularization:

Combination of both L1 and L2 Regularization. Encourages sparsity like L1 but also gives varying weights for features.

Formula: $Loss + \lambda_1 \sum_i w_i^2 + \lambda_2 \sum_i |w_i|$

Use cases: Useful when there are many features and some are redundant or irrelevant but would like to keep some as they are helpful.

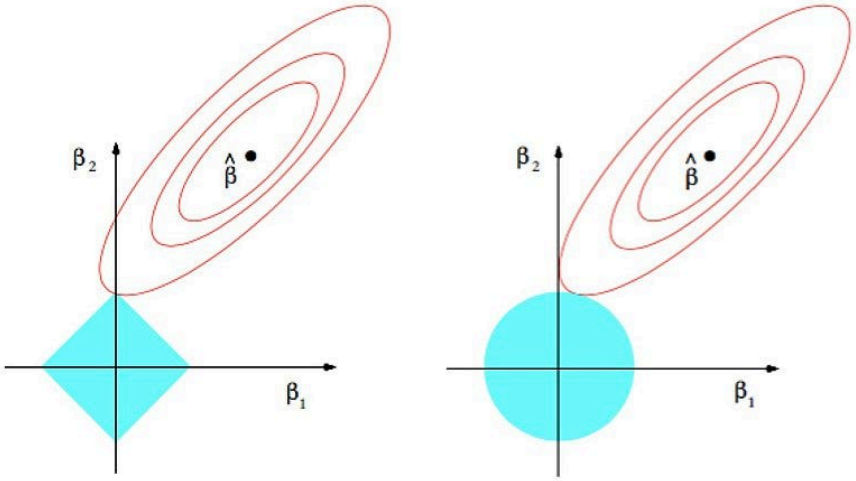


Figure 4.1 L1/L2 regularization functions

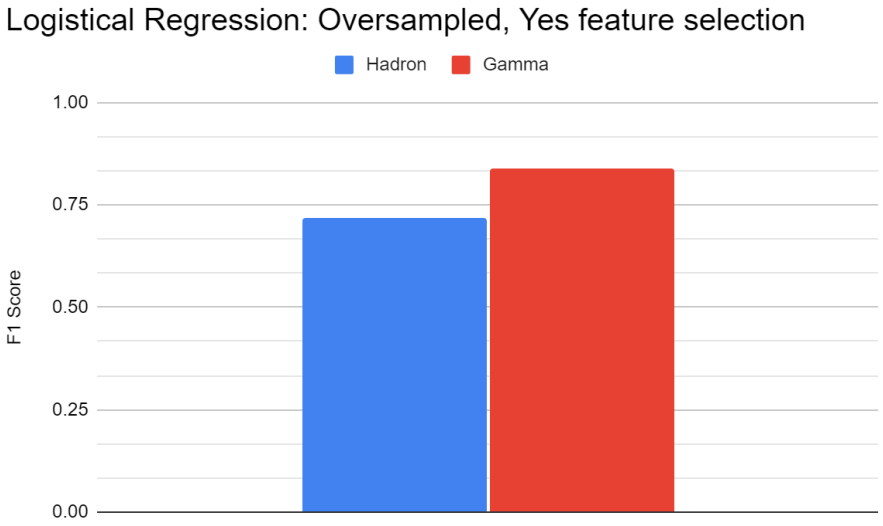


Figure 4.2 F1 score Hadron is 0.72 and Gamma is 0.84

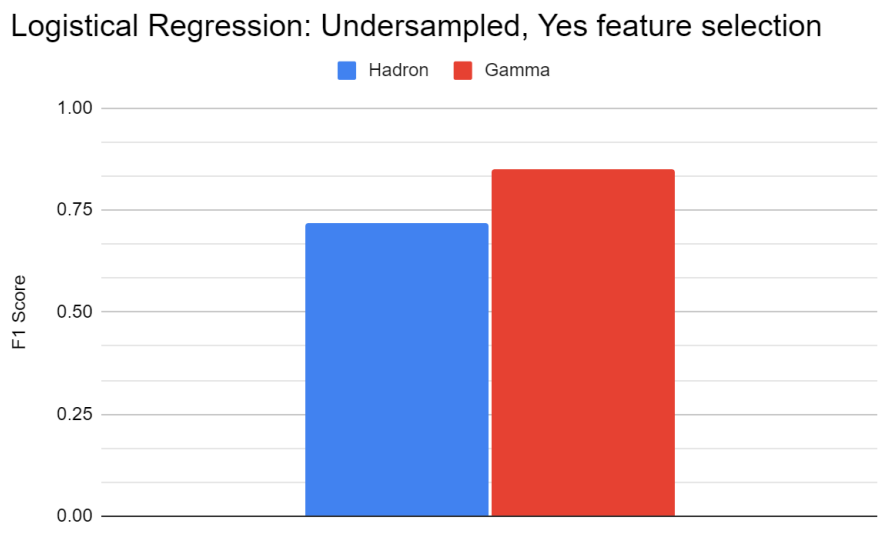


Figure 4.3 F1 score Hadron is 0.72 and Gamma is 0.85

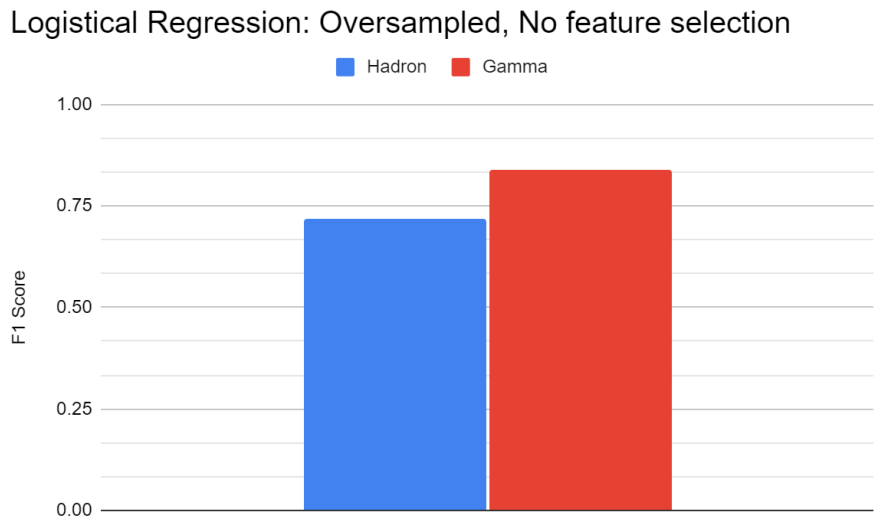


Figure 4.4 F1 score Hadron is 0.72 and Gamma is 0.84

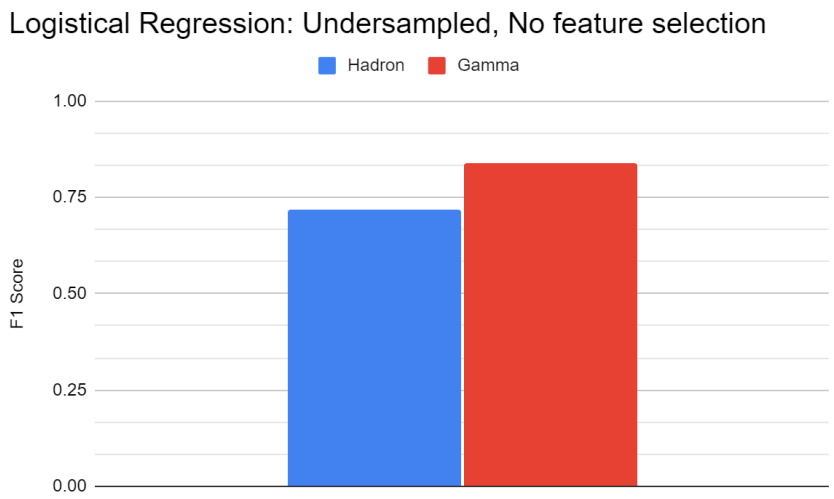


Figure 4.5 F1 score Hadron is 0.72 and Gamma is 0.84

Analysis

F1 scores did not differ at all from undersample and oversampled, nor between feature and not feature selected.

L1/2 Penalties also did not affect the F1 scores, however accuracy of the model did improve from 0.79 to 0.80 when applied L2 with C=0.01. I presume there was no significant difference because the initial features were already very good at predicting the classes.

TEST data results

Oversampled Not feature sampled:

F1 Hadron: 0.69 Gamma: 0.82

Undersampled Not feature sampled:

F1 Hadron: 0.69 Gamma: 0.82

Oversampled feature sampled:

F1 Hadron: 0.69 Gamma: 0.83

Undersampled feature sampled:

F1 Hadron: 0.69 Gamma: 0.83

Support Vector Machine:

Determine a line (2D), plane (3D), and hyperplane (multi-D) that effectively divides the classes (blue on one side and red on the other side). The model will try to look for a line (2D) that would have the maximum margin from closest of the data from the different classes.

Weaknesses: SVM's are not robust to outliers

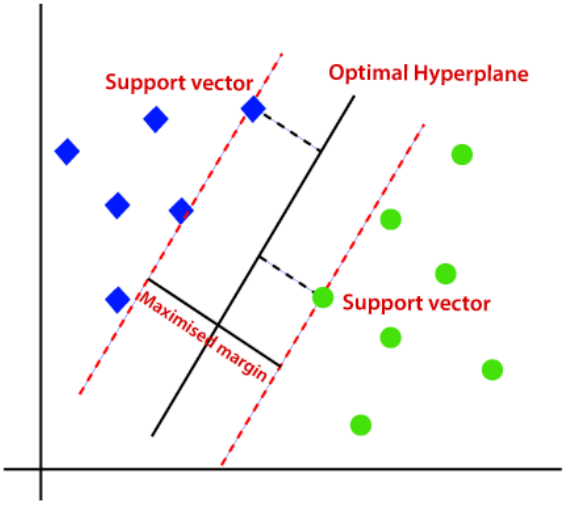


Figure 5.0 Visualization of SVM

Kernel Trick:

If the input is not linearly separable from the original input, we need to apply to transform the data to higher dimensions for them to be now linearly separable.

Kernel Function:

A function that takes as its inputs vectors in the original space and returns the dot product of the vectors.

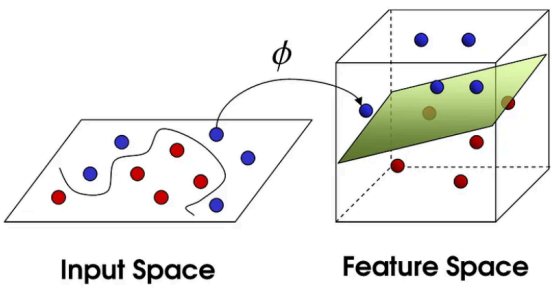


Figure 5.1 Kernal trick

SVM Kernals:

Linear: This is the simplest kernel, and it does not transform the data (equal to directly applying a linear SVM directly to the input data)

Formula: $K(x_i, x_j) = x_i \cdot x_j$ (x's are the input values)

Use case: When data is linearly separable or when the number of features is very large (text classification)

Polynomial: Maps the original feature vectors into a higher-dimensional space where the polynomial combinations of the original features are explicitly considered. It allows the kernel to consider polynomial functions to analyze the data.

Formula: $K(x_i, x_j) = (\gamma x_i \cdot x_j + r)^d$ (γ coefficient that scales the input (default 1), r is the constant term (default 0), d degree of polynomial)

Use case: to capture interactions between features up to a certain degree.

Radial Base function: maps input space into infinite dimensional space. Capable of handling cases where the relationship between class labels and features is nonlinear.

Formula: $K(x_i, x_j) = exp(-\gamma||x_i - x_j||^2)$. γ: Kernel coefficient (determines the radius of influence of a single training example)

Use case: most general problems (when data is not linearly separable)

Sigmoid: related to neural networks, used to model the SVM as a two-layer, perceptron-like neural network.

Formula: $K(x_i, x_j) = tanh(\gamma x_i \cdot x_j + r)$. γ: Kernel coefficient, r: independent term

Use case: contexts that resemble neural network behavior

Figure 5.1 F1 score Hadron: 0.73 Gamma: 0.85

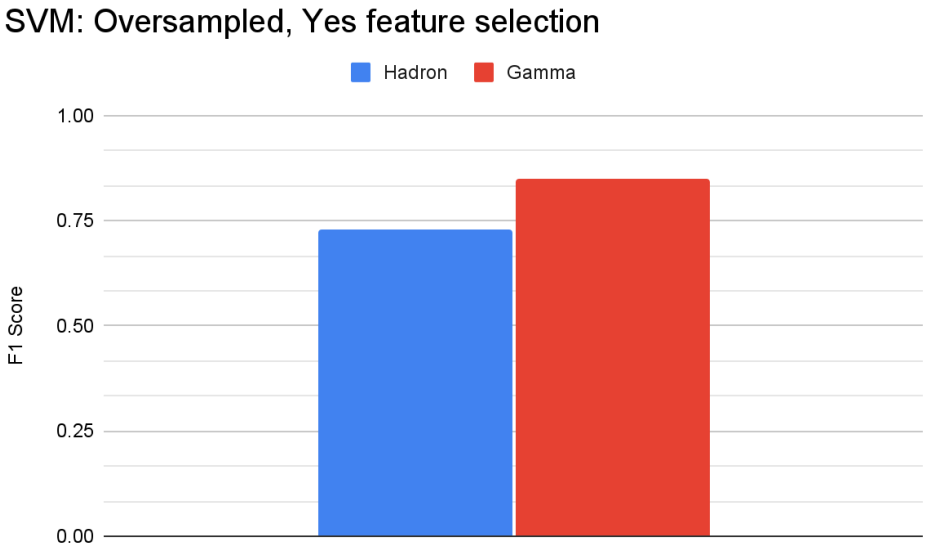


Figure 5.2 F1 score Hadron: 0.73 Gamma: 0.85

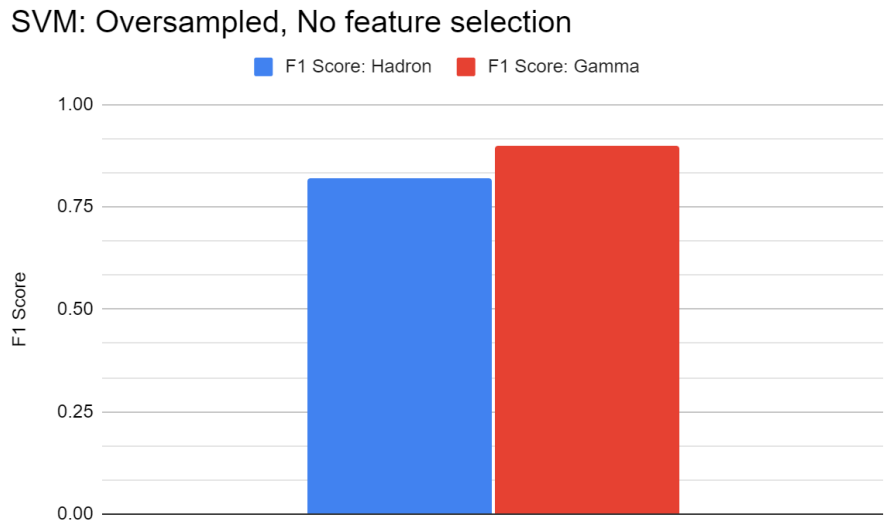


Figure 5.3 F1 score Hadron: 0.82 Gamma: 0.90

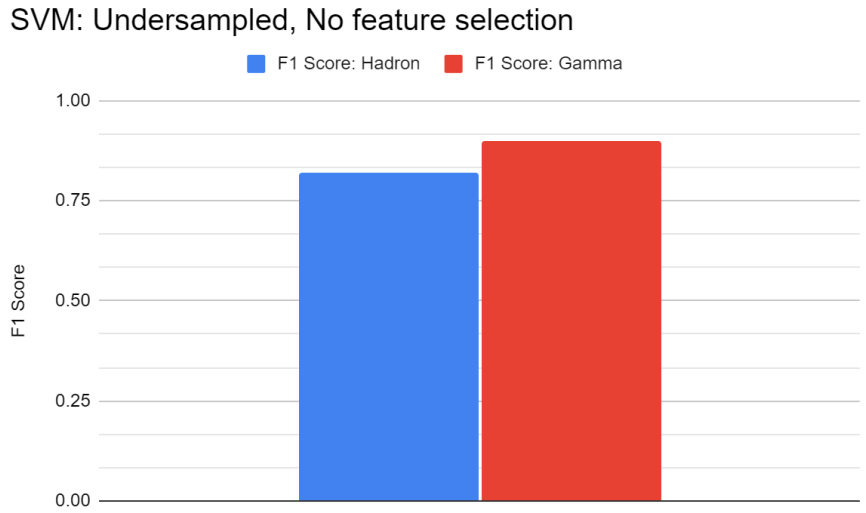


Figure 5.4 F1 score Hadron: 0.81 Gamma: 0.90

Analysis

The not feature-selected model functions much better than the feature-selected one. I presume this to be because of SVM's effectiveness in handling high dimensionality data and loss of useful information when certain features are taken out (less trained on noise so worse at generalizing, but reducing noise can lessen the burden on computational time)

When I tried different Kernals, *rbf* performed the best (0.81,0.90) and *sigmoid* performed the worst (0.54,0.69). This is expected as Sigmoid is mostly suited for neural networks and *rbf* is suited for this project as there are no clear linear divisions.

TEST data results

Oversampled Not feature selected:

F1 Hadron: 0.79 Gamma: 0.89

Neural Networks:

Input Layer: receives the input data and passes it to the next layer. Number of neurons is equal to the number of features in the input data.

Hidden Layer: applies a weighted sum and passes this sum through an activation function.

Weights: determine the strength and direction of the influence one neuron has on another, in the learning process the network adjusts these weights to minimize error.

Biases: better adjust the output data to match the training data, shift the activation function either to the left or right.

Activation Functions: introduces non-linearity into the hidden layer

Sigmoid: maps input values to output values between 0 and 1

ReLU(Rectified Linear Unit): outputs the input directly if positive; otherwise it outputs zero. Helps mitigate the vanishing gradient problem

Tanh: maps values to a range between -1 and 1, leading to better convergence in practice

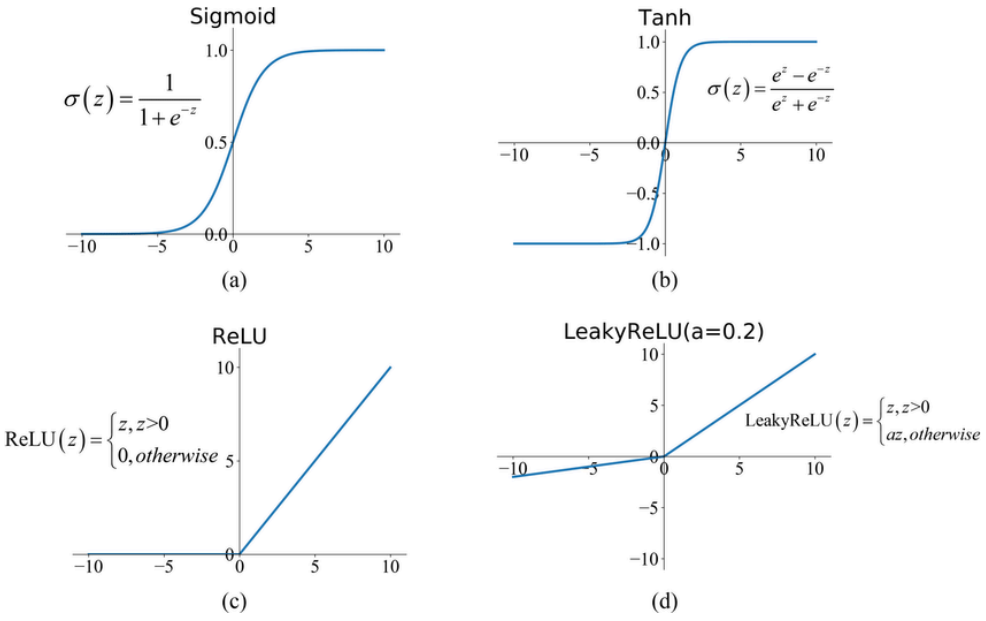


Figure 6.0 Different activation functions

Output Layer: produces the final output, the number of neurons in this layer corresponds to the number of outputs desired.

Forward Propagation: the process of data following from the input>hidden>output
Loss Function: measures the difference between the network prediction and the actual target.

Mean Squared Error: used for regression tasks

Cross Entropy-Loss: used for classification tasks

Backpropagation: the process of adjusting the weights and biases of the network based on the error of the network's predictions

Gradient Descent: gradient (refer to calc 3) points to the steepest increase at a point in a vector field (ie loss function), to decrease the loss we need to move in the opposite direction of the gradient.

Validation_Split: portion of the training data that tensor flow validates the network from and adjusts

Feature Selected Oversample

Variations Tried

- Number of nodes (num_nodes): 16, 32, 64
- Dropout probability (dropout_prob): 0, 0.2
- Learning rate (lr): 0.01, 0.005, 0.001
- Batch size (batch_size): 32, 64, 128

```
nn_model = tf.keras.Sequential([
      tf.keras.layers.Dense(num_nodes, activation='relu',
input_shape=(4,)),
      tf.keras.layers.Dropout(dropout_prob),
      tf.keras.layers.Dense(num_nodes, activation='relu'),
      tf.keras.layers.Dropout(dropout_prob),
      tf.keras.layers.Dense(1, activation='sigmoid')
  ])
```

Figure 6.1 Neural network model used

TEST data results

Legend:

Blue: training data

Orange: validation data

Left graph: Loss function

Right graph: accuracy function

Feature Selected, Oversampled

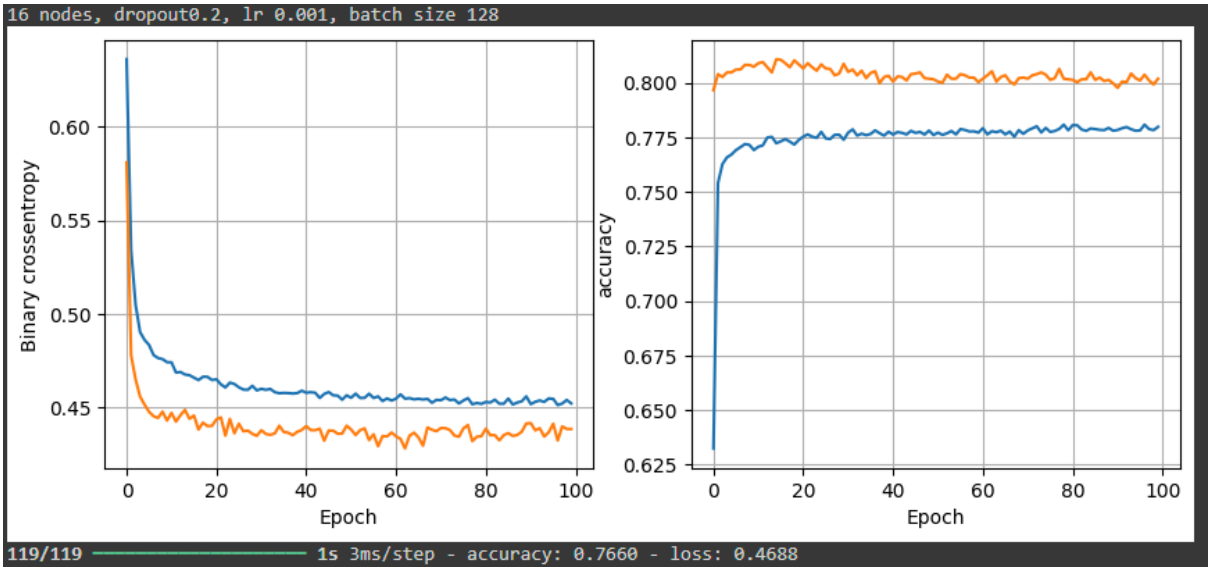


Figure 6.2 Prime example of an overtrained model (16 nodes, dropout_prob 0, lr 0.001, batch_size 128)

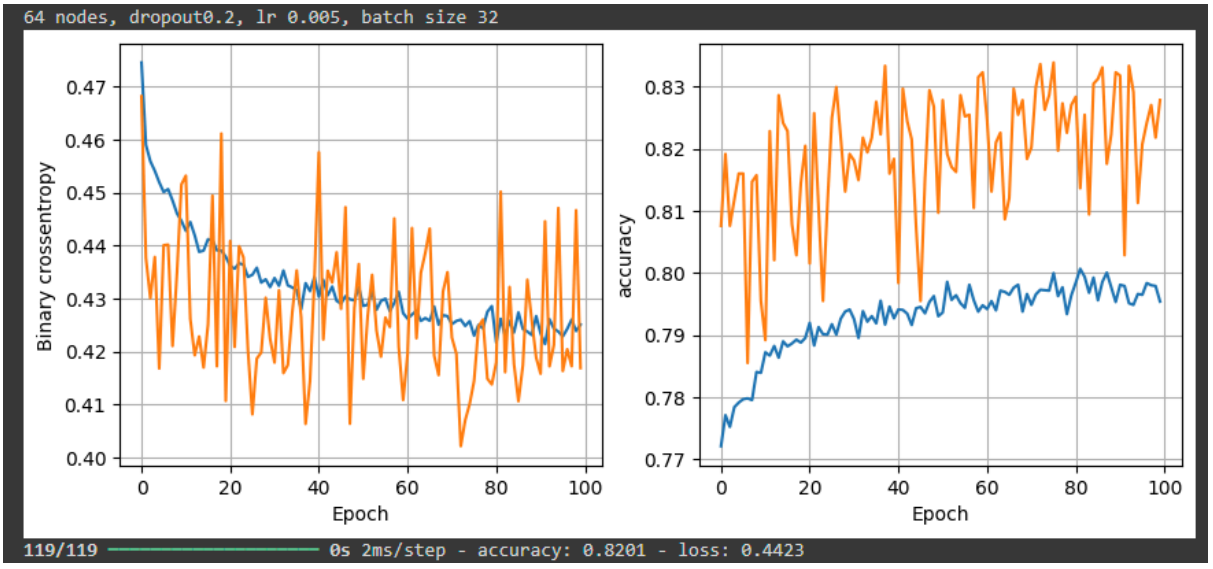


Figure 6.3 Highest accuracy model 0.8201 (64 nodes, dropout_prob 0.2, lr 0.005, batch_size 32)

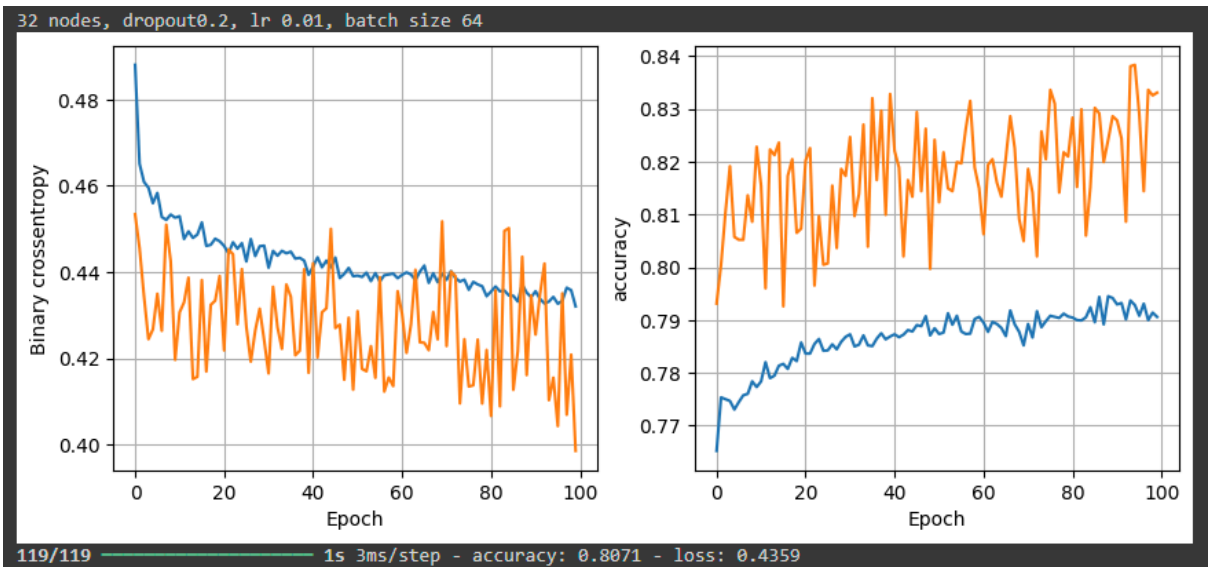


Figure 6.4 Lowest loss model 0.4359 (32 nodes, dropout_prob 0.2, lr 0.01, batch_size 64)

Important Notes

The model is trained only on the train data. As an epoch progresses, the model adjusts its weights and biases to minimize the loss. The validation data trends show the engineer when to stop training the model. For example, if the accuracy/loss improvement becomes stagnant, backpropagation needs to be used to train the model and use the results from the validation data.

Analysis

An anomaly occurs when analyzing the trends, why does the model perform better on the validation data than on the training data? The reason for this is mainly due to one reason, regularization and dropout during training. Regularization and dropout are used to prevent the model from overtraining. These factors can inflate the loss and deflate the accuracy meaning the model is generalized to the training data. So when the model is not applied regularization and dropout during validation it can perform better than expected.

Feature Selected, Undersampled

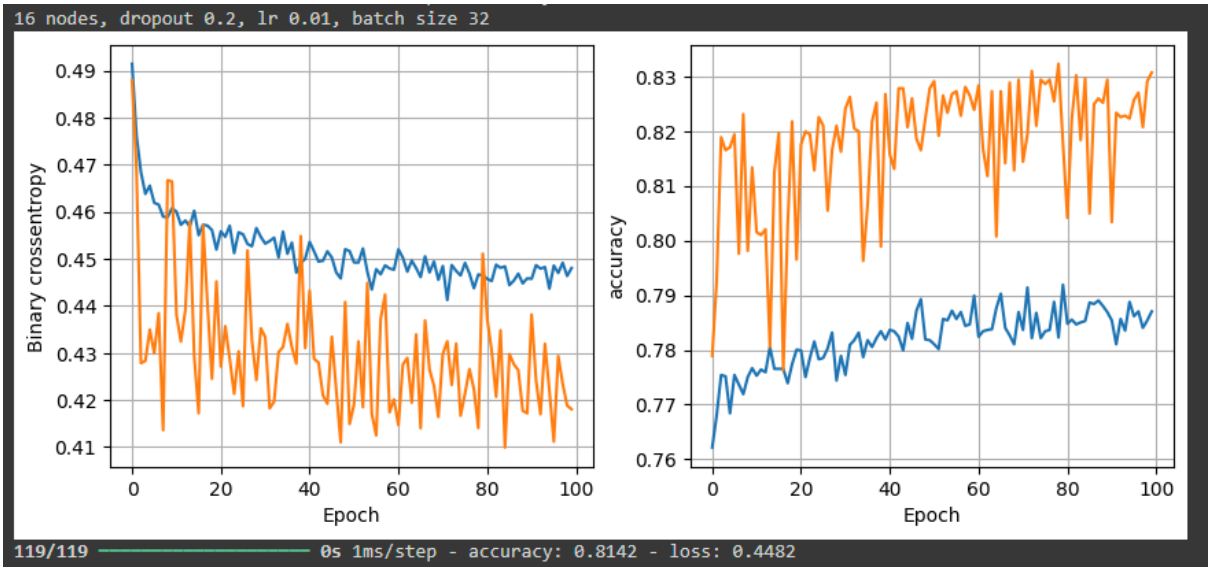


Figure 6.4 Highest Accuracy model 0.8142 (16 nodes, dropout_prob 0.2, lr 0.01, batch_size 32)

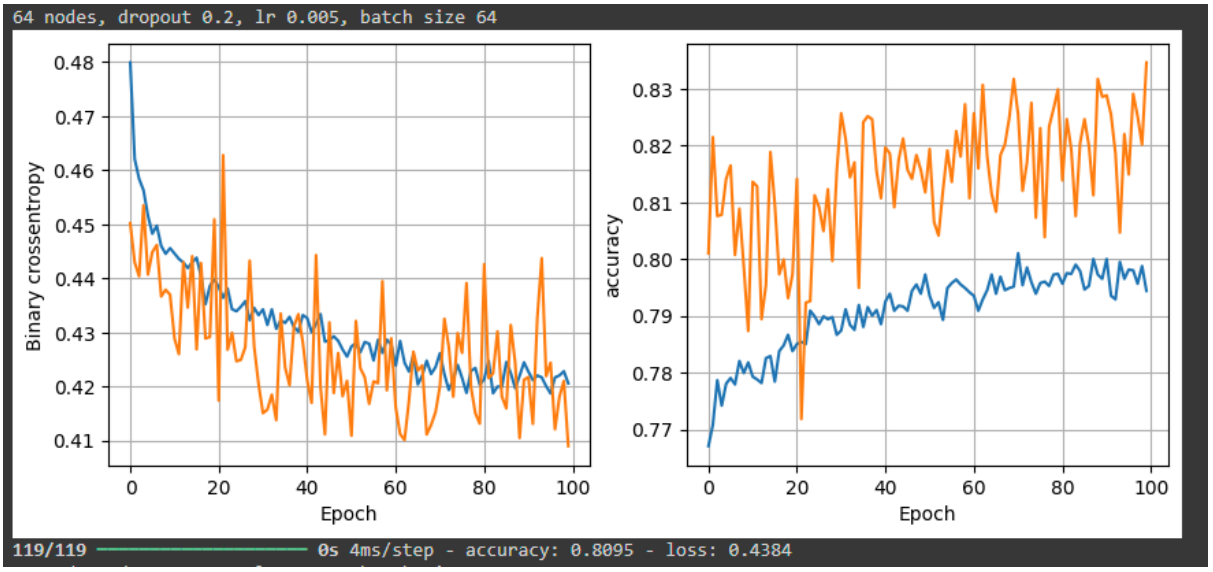


Figure 6.5 Lowest loss model 0.4384 (64 nodes, dropout_prob 0.2, lr 0.005, batch_size 64)

	precision	recall	f1-score	support
0	0.78	0.68	0.73	1331
1	0.84	0.90	0.87	2473
accuracy			0.82	3804
macro avg	0.81	0.79	0.80	3804
weighted avg	0.82	0.82	0.82	3804

Figure 6.6 Least Loss Model Classification Report (0 Hadron, 1 Gamma)

Analysis

Not much significant change is detected, between the oversampled and undersampled, there is not a significant difference between them in terms of accuracy. One thing to note between the validation and train accuracy/loss over epochs is that training exhibits a clear trend of decreasing/increasing (getting better) as the model trains on the data with fewer fluctuations. However, the validation data shows high fluctuation between each epoch showing that the model is somewhat random to the data but a trend is visible such that as the epoch progresses the model is performing better on the validation data (pretty minimal). Due to this I might presume that some fitting to the training data is occurring each run so considering early stopping can be also a method.

Feature Not Selected, Undersampled

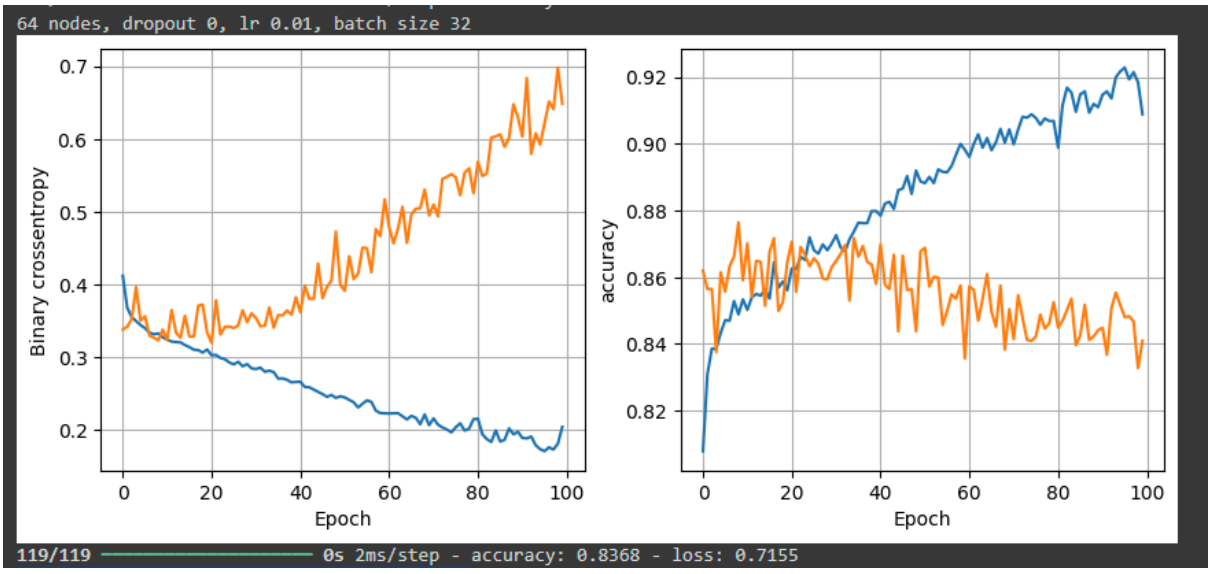


Figure 6.7 Extreme example of over-training).

32 nodes, dropout 0, lr 0.01, batch size 32

```
32 nodes, dropout 0, lr 0.01, batch size 64
32 nodes, dropout 0, lr 0.01, batch size 128
32 nodes, dropout 0, lr 0.005, batch size 32
32 nodes, dropout 0, lr 0.005, batch size 64

64 nodes, dropout 0, lr 0.01, batch size 32
64 nodes, dropout 0, lr 0.01, batch size 64
64 nodes, dropout 0, lr 0.01, batch size 128
64 nodes, dropout 0, lr 0.005, batch size 32
64 nodes, dropout 0, lr 0.005, batch size 64
64 nodes, dropout 0, lr 0.005, batch size 128
```

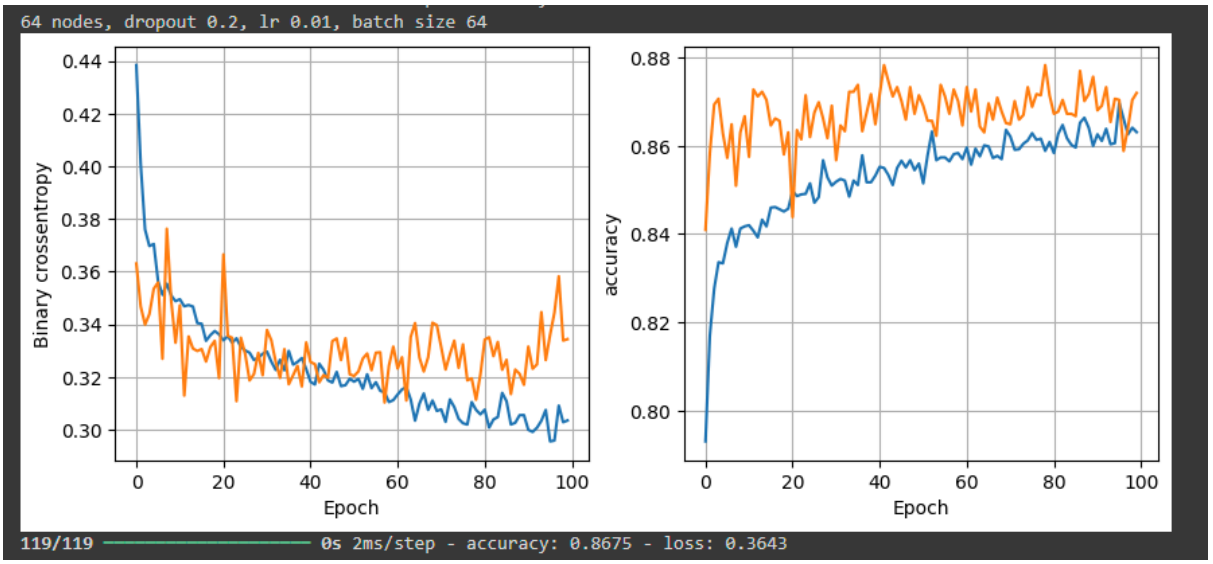
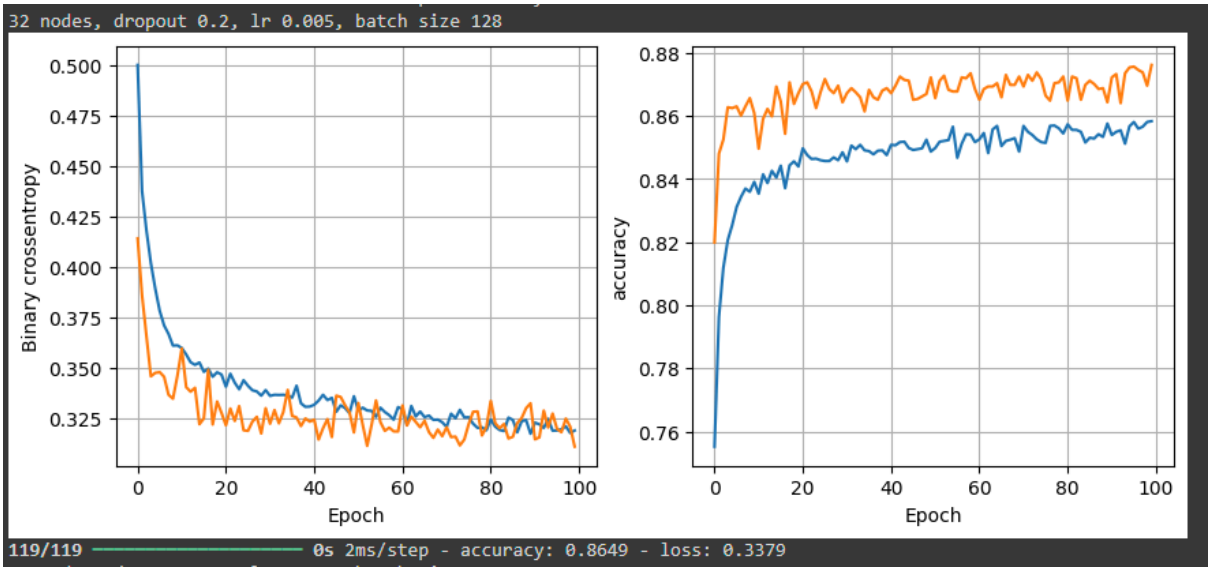


Figure 6.8



	precision	recall	f1-score	support
0	0.85	0.77	0.81	1331
1	0.88	0.92	0.90	2473
accuracy			0.87	3804
macro avg	0.86	0.85	0.86	3804
weighted avg	0.87	0.87	0.87	3804

Feature Not Selected, Oversampled

Would it be better to introduce noise or save image quality? (image recognition project)